

CENTRO UNIVERSITÁRIO UNIFECAP

Cloud DevOps — Orchestrating Containers and Microservices

RENAN GOMES MENDES DE CASTRO

RA: 103686

**Pedidos Veloz — Plataforma de pedidos em microsserviços
do Docker Compose ao Kubernetes com observabilidade e CI/CD**

Parte Prática — Relatório Técnico

São Paulo

2026

SUMÁRIO

Seção	Página
1. Visão geral da solução	3
2. Ambiente local com Docker Compose	3
3. Containerização e versionamento	3
4. Kubernetes — produção mínima	3
5. Service Mesh — Istio	4
6. CI/CD com GitHub Actions	4
7. Estratégia de deploy, escala e observabilidade	4
8. Infraestrutura como Código (Terraform)	5
9. Como executar e validar	5
10. Conclusão	5

1. Visão geral da solução

Este documento descreve a implementação da plataforma Pedidos Veloz para a Loja Veloz, atendendo aos requisitos do desafio: ambiente local automatizado, containerização padronizada, orquestração em Kubernetes, pipeline CI/CD, estratégia de deploy de baixo risco, escalabilidade automática, configuração e credenciais seguras, observabilidade ponta a ponta e infraestrutura reproduzível por IaC. O repositório está estruturado com 4 microsserviços em Python/FastAPI (api-gateway, pedidos, pagamentos, estoque), PostgreSQL como banco e RabbitMQ para mensageria assíncrona, com Istio operando a malha em produção.

2. Ambiente local com Docker Compose

Toda a stack — quatro microsserviços, Postgres, RabbitMQ e o stack de observabilidade — sobe com um único comando, eliminando o problema atual de “cada time sobe como dá em máquinas diferentes”:

```
cp .env.example .env docker compose up --build -d
```

O arquivo docker-compose.yml define a rede backend, volumes nomeados para o Postgres (persistência entre restarts), healthchecks com dependências (depends_on com condition: service_healthy) e variáveis parametrizadas por .env. Um script init-multiple-dbs.sh cria os bancos pedidos e estoque na inicialização. As portas 8080 (gateway), 16686 (Jaeger), 9090 (Prometheus), 3000 (Grafana) e 15672 (RabbitMQ UI) são expostas para inspeção. Instruções e exemplos curl estão no README.

3. Containerização e versionamento

Cada serviço possui um Dockerfile multi-stage idêntico em estrutura: estágio builder compila os wheels Python com pip wheel; estágio runtime instala apenas a partir dos wheels já compilados em uma imagem python:3.12-slim. Boas práticas aplicadas:

- Usuário não-root app (UID 10001) criado com useradd --system.
- readOnlyRootFilesystem habilitado nos manifests K8s (com emptyDir em /tmp).
- HEALTHCHECK nativo do Docker para visibilidade no Compose; probes equivalentes em K8s.
- PYTHONDONTWRITEBYTECODE e PIP_NO_CACHE_DIR para reduzir tamanho.
- Versionamento por tags semver (vX.Y.Z) e SHA curto (sha-abcdef1), aplicadas pelo docker/metadata-action no CI; o registry destino é o GHCR (ghcr.io/loja-veloz/*).

4. Kubernetes — produção mínima

Os manifests estão em k8s/base/ (recurso compartilhado) e k8s/overlays/{dev,prod}/ (variações por ambiente, via Kustomize). O namespace loja-veloz é criado com Pod Security Admission no perfil restricted, garantindo que pods rejeitem privilegeEscalation, hostPath, capabilities adicionais etc. A tabela abaixo resume os recursos do namespace de aplicação:

Recurso	Réplicas (dev/prod)	Notas
Deployment api-gateway	1 / 4	HPA 2 → 10, RollingUpdate maxUnavailable=0
Deployment pedidos	1 / 3	Publica em pedidos.events
Deployment pagamentos-v1	1 / 5	Stable; v2 entra por canary 5%
Deployment estoque	1 / 3	Consumer RabbitMQ + API HTTP
StatefulSet postgres	1	PVC 5Gi; em prod usar RDS gerenciado

Recurso	Réplicas (dev/prod)	Notas
StatefulSet rabbitmq	1	Em prod escalar para 3 com quorum queues

Cada serviço aplicativo tem ServiceAccount próprio, ConfigMap compartilhado (loja-veloz-config) com URLs internas e Secret (loja-veloz-secrets) com credenciais; em produção os Secrets devem vir do External Secrets Operator + AWS Secrets Manager. As readiness/liveness probes verificam respectivamente /health/ready (que checa dependências como banco) e /health/live (apenas processo vivo).

A segurança em camadas é completada por NetworkPolicy default-deny no namespace, com allows explícitos por par (gateway → pedidos, pedidos → postgres, pedidos → rabbitmq etc.), além de PodDisruptionBudget garantindo minAvailable durante drains.

5. Service Mesh — Istio

Istio é injetado via label istio-injection: enabled no namespace. Três configs principais habilitam o modelo de produção:

- **Gateway + VirtualService:** expõem o api-gateway no Ingress da malha em loja-veloz.example.com com retries (3 tentativas, perTryTimeout 2s) e timeout total de 10s.
- **PeerAuthentication STRICT:** exige mTLS entre todos os pods do namespace — zero-trust dentro do cluster, mesmo se um pod for comprometido o tráfego segue criptografado e autenticado por certificado x509 emitido pelo Istio CA.
- **AuthorizationPolicy:** limita quem pode chamar quem por ServiceAccount (ex.: só api-gateway e pedidos podem chamar pagamentos), implementando least privilege na camada L7.
- **Canary:** DestinationRule define os subsets v1/v2 por label e VirtualService aplica peso (95/5 → 75/25 → 0/100). Outlier detection ejeta pods com 5 erros 5xx consecutivos em 30s.

6. CI/CD com GitHub Actions

O workflow .github/workflows/ci.yml executa em três jobs:

- **test:** matrix paralela pelos 4 serviços; instala dependências com cache pip, roda ruff (lint) e pytest --cov, publica coverage como artifact.
- **build:** depende de test; usa docker/build-push-action com cache GHA, faz scan com Trivy (severidade CRITICAL/HIGH, SARIF enviado ao Security tab), aplica tags semver/sha via docker/metadata-action e publica em ghcr.io somente fora de PRs. Permissões mínimas via permissions: no topo.
- **validate-k8s:** kustomize build + kubeconform validam todos os manifests do overlay base; quebra o pipeline se algum schema for inválido.

O workflow cd.yml é disparado pelo CI verde ou manualmente (workflow_dispatch). Ele autentica na AWS via OIDC (sem secret de longo prazo), atualiza tags com kustomize edit set image, aplica via kubectl apply -k e aguarda rollout com kubectl rollout status. Secrets do pipeline são gerenciados em Actions Secrets (escopo de Environment), com aprovação obrigatória para o ambiente prod.

7. Estratégia de deploy, escala e observabilidade

7.1 Deploy

Rolling update é o padrão para todos os serviços stateless — maxSurge: 1, maxUnavailable: 0 garante zero downtime. Para Pagamentos, adota-se canary via Istio: o serviço financeiro tem maior impacto em caso de regressão, então o tráfego é deslocado em três etapas (95/5 → 75/25 → 0/100), observando latência p95 e taxa de erro 5xx em cada etapa. Rollback consiste em reverter os pesos no manifest e fazer apply.

7.2 Escala

HPA configurado para os 4 serviços usando CPU 70% como gatilho principal; behavior.scaleUp permite até +100% de réplicas em 30s para reagir rápido a picos da campanha, enquanto

scaleDown tem janela de estabilização de 5 min para evitar flapping. PodDisruptionBudget complementa: pagamentos exige minAvailable: 2; os demais minAvailable: 1.

7.3 Observabilidade

Os três pilares são integrados via OpenTelemetry SDK em cada serviço:

- Métricas: prometheus-fastapi-instrumentator expõe /metrics; HPA pode futuramente usar requests/s via Prometheus Adapter.
- Logs: stdout em JSON (12-Factor XI); em produção, agregados por Loki + visualizados no Grafana.
- Traces: opentelemetry-instrumentation-fastapi e -httpx propagam o trace context entre os 4 serviços; spans enviados via OTLP HTTP ao OTel Collector e roteados ao Jaeger.

8. Infraestrutura como Código (Terraform)

O diretório terraform/ contém um esqueleto funcional com módulos vpc e eks. O main.tf orquestra a VPC (três subnets privadas e públicas com tags Kubernetes), o cluster EKS 1.30 com encryption-at-rest via KMS e logs do control plane para CloudWatch, e um node group on-demand t3.medium dimensionado por ambiente (dev: 1-4 nós; prod: 3-10 nós). State remoto previsto em S3 com lock em DynamoDB (descomentar no backend após criar os recursos). A justificativa para IaC é reprodutibilidade: cada ambiente é criado e destruído identicamente, com auditoria por commit no Git.

9. Como executar e validar

```
# Local (Docker Compose) cp .env.example .env && docker compose up --build -d # Smoke
test curl -X POST http://localhost:8080/api/v1/pedidos -H "Content-Type:
application/json" \ -d '{"cliente_id":"c1","itens":[{"sku":"SKU-
001","quantidade":2,"preco_unitario":49.90}]}' # Kubernetes (Minikube) minikube start
--cpus=4 --memory=6g kubectl apply -k k8s/overlays/dev kubectl -n loja-veloz get pods -w
kubectl apply -f k8s/istio/
```

O README.md detalha as portas, credenciais default, comandos para testar HPA com carga sintética e como inspecionar traces no Jaeger.

10. Conclusão

A entrega cumpre integralmente os requisitos do desafio: ambiente local com um comando, containerização segura e versionada, manifests K8s prontos para produção, pipeline CI/CD com gates de qualidade e segurança, estratégias de deploy diferenciadas por criticidade (rolling/canary), escalabilidade elástica via HPA, observabilidade nos três pilares, service mesh com mTLS, e infraestrutura reproduzível por Terraform. A arquitetura é alinhada aos princípios cloud-native (CNCF) e ao 12-Factor App, e segue como referência a arquitetura pública Google Cloud Online Boutique. O resultado é uma plataforma capaz de absorver o pico da campanha promocional da Loja Veloz com risco reduzido e tempo de diagnóstico curto em caso de falhas.